

Predavanje 9_b

Poznato je da ukoliko imamo više primjeraka iste strukture ili klase, tada u normalnim okolnostima svaki primjerak ima *zasebne primjerke* svih svojih atributa. Recimo, ukoliko su "d1" i "d2" dvije promjenljive tipa "Datum" gdje je "Datum" neka struktura ili klasa koja ima atribut nazvan "dan", tada se "d1.dan" i "d2.dan" ponašaju kao dvije *potpuno odvojene promjenljive*. Međutim, nekada je potrebno imati attribute koji će zajednički dijeliti *svi primjerci iste klase* i koji će, na neki način, predstavljati svojstvo same klase, a ne individualnih primjeraka te klase. Takvi atributi nazivaju se *statički atributi* i deklariraju se uz pomoć modifikatora "static". Neka, na primjer, imamo klasu "Student" opisanu atributima "ime", "prezime", "indeks", "ocjene" i "prosjeck". Dalje, neka je potrebno imati informaciju o *ukupnom broju studenata*, pri čemu ta informacija treba biti dostupna svim primjercima klase "Student", bez obzira gdje se oni nalazili unutar programa. Također, pretpostavimo da samo metode klase "Student" trebaju pristupati ovoj informaciji. Jedno očigledno rješenje je definirati *globalnu promjenljivu* "broj_studenata" koja će čuvati traženu informaciju, ali njoj će tada moći pristupati svako, i ko treba i ko ne treba, tako da postoji mogućnost nehotičnog (ili namjernog) neovlaštenog pristupa. Mnogo bolja ideja je definirati da "broj_studenata" bude atribut klase "Student". Međutim, kada bi to bio obični atribut, tada bi svaki primjerak klase "Student" mogao postaviti različite vrijednosti ovog atributa, iako je jasno da je ukupan broj studentata svojstvo koje je zajedničko za sve primjerke klase "Student". Pravo rješenje je definirati "broj_studenata" kao *statički atribut klase* "Student", kao u sljedećoj deklaraciji:

```
class Student {  
    static int broj_studenata;           // Statički atribut...  
    std::string ime, prezime;  
    int indeks;  
    std::array<int, 50> ocjene;  
    double prosjek;  
public:  
    ...                                // Deklaracije metoda nam zasad nisu bitne...  
};
```

Ukoliko sad pretpostavimo da su "s1" i "s2" dva primjerka ovako deklarirane klase "Student", tada se izrazi "s1.broj_studenata" i "s2.broj_studenata" ne ponašaju kao dvije različite, nego kao *jedna te ista promjenljiva*, odnosno svi primjerci klase "Student" dijele istu vrijednost atributa "broj_studenata" i njegova eventualna promjena izvršena iz neke metode klase "Student" primijenjene nad nekim konkretnim objektom biće vidljiva svim drugim primjercima iste klase. Atribut "broj_studenata" se u potpunosti ponaša kao *globalna promjenljiva*, osim što se može koristiti samo na mjestima gdje su privatni atributi klase "Student" dostupni (tj. unutar metoda ove klase i njenih prijateljskih funkcija). Međutim, deklaracija statičkog atributa unutar klase samo *najavljuje* njegovo postojanje, ali ga i ne *definira* (odnosno, ne rezervira memorijski prostor za njegovo pamćenje). Svaki statički atribut se *mora na nekom mjestu u programu posebno definirati na globalnom nivou*, dakle izvan definicije funkcija ili klase (pri čemu se tom prilikom može eventualno izvršiti njegova inicijalizacija), deklaracijom poput sljedeće:

```
int Student::broj_studenata = 10;      // Tek sada atribut zaista postoji!
```

Razlog za ovu zavrzlamu je tehničke prirode i vezan je za činjenicu da se C++ programi mogu podijeliti u više različitih programskih datoteka koje se mogu neovisno kompajlirati i na kraju povezati u jedinstven program, ali u detalje ovdje ne možemo ulaziti. Ipak, bez obzira na činjenicu da se statički atributi definiraju izvan deklaracije klase, poput definicije običnih globalnih promjenljivih, pravo pristupa im je strogo ograničeno, kao što smo već objasnili.

Statički atributi se gotovo isključivo deklariraju u privatnoj sekciji klase (zapravo, načela objektno orijentiranog programiranja nalažu ovo za sve attribute, a ne samo za statičke). U principu, nije nemoguće izvršiti deklaraciju statičkih atributa u javnoj sekciji klase, ali se na taj način donekle gubi smisao njihovog uvođenja. Na primjer, pretpostavimo da je statički atribut "broj_studenata" deklariran u javnoj sekciji klase "Student" i da su "s1" i "s2" dva konkretna primjerka klase "Student". Tada bismo atributu "broj_studenata" mogli pristupiti iz *bilo kojeg dijela programa koji se nalazi iza mjesta njegove deklaracije* konstrukcijama poput "s1.broj_studenata" ili "s2.broj_studenata" (u oba slučaja pristup se odnosi na *istu* jedinku). Čak bi bio legalan i pristup pomoću izraza "Student::broj_studenata" izvan neke od metoda klase "Student", jer je *svejedno* na koji se konkretan primjerak klase "Student" ovaj atribut odnosi, s obzirom da je on zajednički za sve primjerke klase (ovo je dozvoljeno samo za statičke attribute). Drugim riječima, ovaj atribut bi se mogao koristiti isto kao i ma kakva globalna promjenljiva, jedino bi nas sintaksa upućivala da se radi o podatku koji je na neki način logički vezan za klasu "Student" (u

striktno objektno orijentiranim programskim jezicima kao što su npr. Java i C#, statički atributi s javnim pravima pristupa koriste se za simulaciju globalnih promjenljivih, koje u takvim jezicima ne postoje).

Funkcije članice također mogu biti statičke. Kao što su statički atributi u suštini klasične globalne promjenljive, samo s ograničenim pravima pristupa, tako su i statičke funkcije članice zapravo klasične funkcije s ograničenim pravima pristupa. Unutar statičkih funkcija članica ne postoji pokazivač "**this**", tako da statičke funkcije članice *ne mogu znati* nad kojim su objektom pozvane. Zbog toga, one ne mogu direktno pristupati atributima objekta nad kojim su pozvane, osim eventualno statičkim atributima (koji su zajednički za sve primjerke iste klase), niti mogu pozivati samostalno (tj. bez navođenja imena konkretnog objekta) druge funkcije članice klase, osim eventualno druge statičke funkcije članice. Za njih je zapravo *posve nebitno* nad kojim se objektom pozivaju. One se koriste kada želimo implementirati neku akciju koja ne ovisi od toga nad kojim je konkretnim objektom pozvana, a ne želimo za tu svrhu upotrijebiti klasičnu funkciju, s obzirom da smatramo da je ta akcija po svojoj prirodi ipak tijesno vezana za samu klasu, ili kada tu akciju želimo "sakriti od javnosti" (stavljanjem u privatni dio klase). Isto tako, statičke funkcije članice mogu se koristiti i za manipulaciju (čitanje ili modifikaciju) sa statičkim atributima klase. Pored toga, statičke funkcije članice također imaju pravo pristupa čak i nestatičkim privatnim članovima primjeraka klase kojoj pripadaju, pod uvjetom da se navede *na koji konkretan objekat* se ti članovi odnose. Uskoro ćemo vidjeti da se ova osobina može jako korisno upotrijebiti.

Razmotrimo statičke funkcije članice na konkretnom primjeru klase "**Datum**" koju smo ranije razvili. Ukoliko uporedimo metode "**Postavi**" i "**PredjiNaSljedeciDan**", primjećujemo da se u obje metode javlja potreba za utvrđivanjem broja dana u određenom mjesecu, zbog čega se u obje metode javlja ista sekvenca naredbi. Dupliranje kôda bismo mogli izbjeći ukoliko bismo definirali posebnu funkciju nazvanu recimo "**BrojDanaUMjesecu**" koja bi vraćala broj dana u mjesecu koji se prenosi kao parametar (pri tome bismo morali kao parametar prenositi i godinu, jer od nje može zavisiti koliko dana ima februar). Funkcija "**BrojDanaUMjesecu**" bi, sasvim lijepo, mogla biti klasična funkcija. Međutim, kako je ona razvijena za potrebe klase "**Datum**", prirodnije bi bilo da ona bude *sastavni dio klase*, odnosno da bude njena funkcija članica. Dalje, kako ova funkcija ne treba pristupati atributima klase (s obzirom da ona sve neophodne informacije dobija putem parametara), možemo je učiniti *statičkom funkcijom članicom*. I zaista, potpuno je svejedno nad kojim će se objektom funkcija "**BrojDanaUMjesecu**" pozvati (npr. broj dana u martu je uvijek 31, neovisno od razmatranog datuma). Pored toga, proglašavanje metode "**BrojDanaUMjesecu**" za statičku metodu čini je i neznatno efikasnijom, jer se u tom slučaju njoj ne prenosi pokazivač "**this**" kao skriveni parametar. Stoga ćemo izmijeniti deklaraciju klase "**Datum**" da izgleda ovako (pri tome, dio deklaracije koji ostaje isti nismo ponovo prepisivali, radi uštede prostora):

```
class Datum {
    int dan, mjesec, godina;
    const char *DajImeMjeseca() const;
    static int BrojDanaUMjesecu(int mjesec, int godina);
public:
    ...
    // Javni dio ostaje isti kao i ranije...
};
```

Statičku metodu "**BrojDanaUMjesecu**" smo stavili u privatnu sekciju klase, čime je onemogućeno da se ona poziva izvan funkcija članica klase. Njena implementacija, kao i nove izvedbe metoda "**Postavi**" i "**PredjiNaSljedeciDan**", koje se oslanjaju na njeno postojanje, mogle bi izgledati recimo ovako (ovdje je određivanje broja dana izvedeno na drugačiji način nego ranije, čisto radi ilustracije različitih ideja):

```
int Datum::BrojDanaUMjesecu(int mjesec, int godina) {
    if(mjesec == 4 || mjesec == 6 || mjesec == 9 || mjesec == 11) return 30;
    if(mjesec != 2) return 31;
    if(godina % 4 == 0 && godina % 100 != 0 || godina % 400 == 0) return 29;
    return 28;
}

void Datum::Postavi(int dan, int mjesec, int godina) {
    if(godina < 1 || dan < 1 || mjesec < 1 || mjesec > 12
        || dan > BrojDanaUMjesecu(mjesec, godina))
        throw std::domain_error("Neispravan datum!");
    Datum::dan = dan; Datum::mjesec = mjesec; Datum::godina = godina;
}

void Datum::PredjiNaSljedeciDan() {
    dan++;
    if(dan > BrojDanaUMjesecu(mjesec, godina)) dan = 1, mjesec++;
    if(mjesec > 12) mjesec = 1, godina++;
}
```

U suštini, s aspekta funkcioniranja, sve bi isto radilo i da metoda "**BrojDanaUMjesecu**" nije bila statička (osim što bi se bespotrebno prenosio i pokazivač "**this**"). Međutim, prirodnije ju je definirati kao statičku, s obzirom da ona zaista ne mora znati nad kojim objektom djeluje. Također, postoje i neki konteksti u kojima je moguće primijeniti samo statičke metode, a ne i nestatičke, tako da je dobro da se odmah naviknemo na njihovu upotrebu. Recimo, pokazivači na funkcije mogu bez problema pokazivati na statičke metode (pod uvjetom da imaju odgovarajući tip i broj parametara), ali *ne i na obične (nestatičke) metode* (zbog nemogućnosti ažuriranja "**this**" pokazivača putem pokazivača na funkciju).

S obzirom da je metoda "**BrojDanaUMjesecu**" definirana kao privatna, njeno pozivanje izvan funkcija članice klase nije dozvoljeno. Da je definirana kao javna, ona bi se mogla pozivati iz bilo kojeg dijela programa koji slijedi iza deklaracije klase konstrukcijama poput "**dat.BrojDanaUMjesecu(2, 2005)**" gdje je "**dat**" neka konkretna promjenljiva tipa "**Datum**" (koja zapravo ni na koji način ne utiče na samo izvršavanje funkcije), odnosno konstrukcijama poput "**Datum::BrojDanaUMjesecu(2, 2005)**" s obzirom da je posve nebitno nad kojim je konkretnim primjerkom klase "**Datum**" metoda pozvana (naravno, ovo je moguće samo za statičke funkcije članice). Dakle, tada bi se ova metoda mogla koristiti posve identično kao i obična (globalna) funkcija, samo bi nas sintaksni prefiks "**Datum::**" upućivao da se radi o funkciji čija je svrha postojanja tijesno vezana za klasu "**Datum**" (inače, kako u striktno objektno orijentiranim jezicima kao što su Java i C# obične globalne funkcije uopće ne postoje, nego samo funkcije članice, statičke funkcije članice se u takvim jezicima često koriste upravo za simulaciju običnih funkcija).

Unutar deklaracija klase mogu se naći i deklaracije *korisnički imenovanih tipova* ("**typedef**" ili "**using**" deklaracije), *pobrojanih tipova* ("**enum**" deklaracije), pa čak i deklaracije drugih struktura ili klasa. U slučaju deklaracije jedne strukture ili klase unutar deklaracije druge klase govorimo o tzv. *ugniježdjenim strukturama ili klasama* (engl. *nested structs/classes*). Svim ovakvim tvorevinama unutar funkcija članica klase možemo pristupiti na uobičajeni način, kao da su deklarirane izvan klase. Međutim, njihovo korištenje izvan funkcija članica klase (ili prijateljskih funkcija) moguće je samo ukoliko su deklarirane u javnoj sekciji klase, i to uz obavezno navođenje imena pripadne klase i operatora "**::**". Na primjer, djeluje razumno u klasu "**Datum**" dodati i sljedeće deklaracije:

```
class Datum {  
    ... // Isto kao i ranije...  
public:  
    enum Mjeseci {Januar, Februar, Mart, April, Maj, Juni, Juli, August, Septembar,  
        Oktobar, Novembar, Decembar};  
    enum Dani {Ponedjeljak, Utorak, Srijeda, Cetvrtak, Petak, Subota, Nedjelja};  
    ... // Isto kao i ranije...  
};
```

Uz ovakvu deklaraciju, tipove "**Mjeseci**" i "**Dani**", kao i pobrojane konstante ovih tipova (poput "**Maj**", "**Petak**", itd.) unutar funkcija članica klase možemo koristiti na isti način kao i da su deklarirani izvan klase. Međutim, ukoliko im želimo pristupiti iz drugih dijelova programa, moramo koristiti konstrukcije poput "**Datum::Mjeseci**", "**Datum::Dani**", "**Datum::Maj**", "**Datum::Petak**", itd. (s ovim smo se već susreli kod konstrukcije "**enum class**", što objašnjava i sam naziv "**enum class**"). Slično vrijedi i za ugniježdene strukture ili klase. Recimo, da klasa "**Datum**" ima ugniježdenu klasu nazvanu "**Ugniježdjena**", njoj bismo izvan klase pristupili pomoću konstrukcije "**Datum::Ugniježdjena**", tako da bismo promjenljivu "**u**" tipa te klase deklarirali pomoću deklaracije "**Datum::Ugniježdjena u;**" (recimo, svi tipovi koji podržavaju iteratore, imaju definirane iteratore kao svoje ugniježdene klase).

U prethodnom primjeru, pristup pobrojanim tipovima "**Mjeseci**" i "**Dani**" i njihovim odgovarajućim pobrojanim konstantama iz drugih dijelova programa, moguć je samo zahvaljujući činjenici da smo njihove deklaracije smjestili u javnu sekciju klase "**Datum**". U suprotnom, svi ovi identifikatori bili bi nedostupni izvan metoda i funkcija prijatelja klase. Slijedi da je deklariranje korisnički imenovanih i pobrojanih tipova koji se *samo interno koriste za potrebe implementacije klase*, a za koje *ostatak programa ne treba da zna*, najbolje izvesti upravo u privatnoj sekciji klase. Isto vrijedi i za razne pomoćne strukture odnosno klase, koje se samo interno koriste za potrebe implementacije neke druge klase.

Objektno zasnovani pristup razvoju demonstriraćemo na još jednoj interesantnoj klasi koju ćemo razviti. Pretpostavimo da nam je potrebna klasa "**Vektor3d**" koja će opisivati vektor u prostoru, definiran s tri koordinate "**x**", "**y**" i "**z**". One će ujedno biti i atributi ove klase. U skladu s principima objektno orijentiranog dizajna, ovi atributi će biti *privatni* (bez obzira što u ovom slučaju ima smisla postaviti *bilo koji atribut na bilo koju vrijednost*), pri čemu ćemo uvesti metode "**Postavi**", "**Ocitaj**", "**DajX**", "**DajY**" i "**DajZ**" koje su analogne sličnim metodama klase "**Datum**" (između ostalog, na ovaj način onemogućavamo da se koordinate vektora postavljaju odvojeno jedna od druge, već samo sve tri skupa,

što je mnogo smislenije). Pored toga, uvešćemo i metodu "Ispisi" koja ispisuje vektor kao skupinu od tri koordinate unutar vitičastih zagrada razdvojene zarezima, zatim metodu "DajDuzinu" koja vraća kao rezultat dužinu vektora, kao i metode "MnoziSaSkalarom" i "Saberisa" koje množe vektor sa skalarom (zadanim kao parametar) odnosno sabiraju vektor s drugim vektorom (također zadanim kao parametar). Na taj način dobijamo sljedeću deklaraciju klase, u kojoj su sve metode dovoljno jednostavne da mogu biti implementirane unutar same deklaracije klase:

```
class Vektor3d {
    double x, y, z;
public:
    void Postavi(double x, double y, double z) {
        Vektor3d::x = x; Vektor3d::y = y; Vektor3d::z = z;
    }
    void Ocitaj(double &x, double &y, double &z) const {
        x = Vektor3d::x; y = Vektor3d::y; z = Vektor3d::z;
    }
    void Ispisi() const { std::cout << "{" << x << ", " << y << ", " << z << "}"; }
    double DajX() const { return x; }
    double DajY() const { return y; }
    double DajZ() const { return z; }
    double DajDuzinu() const { return std::sqrt(x * x + y * y + z * z); }
    void PomnoziSaSkalarom(double s) { x *= s; y *= s; z *= s; }
    void Saberisa(const Vektor3d &v) { x += v.x; y += v.y; z += v.z; }
};
```

Primjer je dovoljno jasan da ne traži posebna objašnjenja. Interesan je jedino način na koji se koriste metode "PomnoziSaSkalarom" i "Saberisa", koji je ilustriran u sljedećem primjeru upotrebe klase "Vektor3d" (koji će na ekranu ispisati "7.071068", "{15,20,25}" i "{22,23,27}"), iz čega vidimo da se ove dvije metode na izvjestan način ponašaju poput operatora "*" i "+" (što bi, između ostalog, trebalo da bude jasno i iz načina kako su one definirane):

```
Vektor3d v1, v2;
v1.Postavi(3, 4, 5);
v2.Postavi(7, 3, 2);
std::cout << v1.DajDuzinu() << std::endl;           // 7.071068
v1.PomnoziSaSkalarom(5);
v1.Ispisi();                                           // {15,20,25}
std::cout << std::endl;
v1.Saberisa(v2);
v1.Ispisi();                                           // {22,23,27}
```

Može se primijetiti da smo i u ovom slučaju metodu "DajDuzinu" koja vraća dužinu vektora imenovali na isti način kao i pristupne metode poput "DajX", čime ostvarujemo uniformirani pristup informacijama o primjercima klase "Vektor3d". Na taj način, korisnik klase nema potrebu da zna da dužina vektora nije neki od njegovih atributa, nego svojstvo koje se indirektno izvodi iz koordinata vektora (koje *jesu* njegovi atributi). I zaista, sasvim bi bilo moguće zamisliti implementaciju klase "Vektor3d" u kojoj koordinate ne bi bile atributi, nego bi atributi bili recimo dužina vektora i uglovi koje vektor zaklapa s koordinatnim osama (tzv. *Eulerovi uglovi*), pri čemu bi metode poput "DajX" računale odgovarajuće koordinate iz raspoloživih atributa (uz malu pomoć trigonometrije), dok bi vraćanje dužine postalo trivijalno. Mada bi se implementacija takve klase značajno razlikovala od ovdje date implementacije, korisnik bi tako implementiranu klasu koristio na isti način kao i ovdje razvijenu klasu!

Bitno je naglasiti da princip skrivanja podataka i enkapsulacije strogo naređuje da, osim ukoliko ne postoji veoma jak razlog za suprotno, *sve attribute klase uvijek treba proglasiti za privatne* (postavljanje atributa u javni dio klase se eventualno dopušta samo u postupku debugiranja). Na primjer, čak i ukoliko želimo da imamo mogućnost neovisnog postavljanja koordinata vektora, to *ne treba učiniti tako što ćemo attribute "x", "y" i "z" prosto proglasiti javnim*. Umjesto toga, potrebno je uvesti tri nove trivijalne metode "PostaviX", "PostaviY" i "PostaviZ" (dakle, tri nova *postavljača* ili *setera*) koje će respektivno postavljati vrijednosti atributa "x", "y" odnosno "z" na vrijednost zadanu parametrom (na engleskom govornom području, metode tog tipa gotovo uvijek dobijaju imena koja počinju sa "Set", recimo "SetX"). Tako ćemo, ukoliko na primjer želimo da postavimo "x" koordinatu vektora "v" na vrijednost 5, umjesto "v.x = 5" pisati "v.PostaviX(5)". Mada ovo djeluje kao potpuno suvišna komplikacija, *postoje veoma jaki razlozi za ovakvu praksu*. Zaista, pretpostavimo da se u budućnosti pojavi potreba da u potpunosti promijenimo internu organizaciju atributa vektora "v", i da npr. umjesto u tri atributa realnog tipa koordinate čuvamo u atributu tipa niza od 3 realna elementa (nazvanog recimo "koordinata"). Tada bismo u slučaju da su atributi bili javni, svaki pristup tipa "v.x" morali zamijeniti pristupom poput

"`v.koordinate[0]`" (na primjer, "`v.koordinate[0] = 5`"). S druge strane, ukoliko imamo metode poput "`PostaviX`", dovoljno je samo promijeniti njihovu implementaciju. U ostatku programa se i dalje može bez problema koristiti konstrukcija poput "`v.PostaviX(5)`". Slična stvar bi se dogodila i kada bismo recimo izmijenili implementaciju klase tako da se kao atributi vektora umjesto koordinata čuva dužina vektora i uglovi koje vektor zaklapa s koordinatnim osama, ili ukoliko bismo na bilo koji drugi način izmijenili internu organizaciju klase "`Vektor3d`". Skrivanje atributa omogućava kreatoru klase da po volji mijenja način *funkcioniranja* klase, a da se to ni na koji način ne odrazi na način njene *upotrebe*!

Neosporno je da objektno zasnovani pristup traži *više pisanja* u odnosu na klasični pristup, pogotovo u slučaju manjih programa. Međutim, takav pristup *nije ni namijenjen za pisanje malih programa*. U objektno zasnovanom pristupu se mnogo više *misli na budućnost* i na činjenicu da će se svaki program koji ičemu vrijedi prije ili kasnije morati prepravljati, nadograđivati i usavršavati. Objektno zasnovano i objektno orijentirano programiranje nam priprema teren s ciljem da eventualne kasnije intervencije na programu budu što je god moguće bezbolnije.

U klasi "`Vektor3d`", metode "`PomnoziSaSkalarom`" i "`SaberiSa`" napisane su kao metode koje ne vraćaju nikakav rezultat, što može djelovati sasvim prirodno. Nažalost, zbog toga neke konstrukcije poput "`v2 = v1.PomnoziSaSkalarom(5)`" nisu izvodljive (s obzirom da metoda "`PomnoziSaSkalarom`" ne vraća ništa kao rezultat što bismo mogli pridružiti drugom vektoru), mada djeluju prirodno. Stoga bismo interesantan efekat mogli ostvariti ukoliko bismo izmijenili ove dvije metode tako da kao rezultat *vraćaju izmijenjeni vektor*. Tako bi konstrukcije poput "`v1.PomnoziSaSkalarom(5)`" i dalje ostale legalne (s obzirom da vraćeni rezultat uvijek možemo ignorirati), ali bi postale ispravne i konstrukcije poput "`v2 = v1.PomnoziSaSkalarom(5)`" pa čak i razne lančane (kaskadne) konstrukcije kao što je recimo "`v1.PomnoziSaSkalarom(5).Ispisi()`". Zaista, poziv poput "`v1.PomnoziSaSkalarom(5)`" će kao rezultat vratiti objekat tipa "`Vektor3d`", na koji se onda može primijeniti metoda "`Ispisi`". Tako dobijamo sljedeću verziju klase "`Vektor3d`":

```
class Vektor3d {
...           // Ovdje treba umetnuti definicije atributa i preostalih metoda...
    Vektor3d PomnoziSaSkalarom(double s) {
        x *= s; y *= s; z *= s;
        return *this;
    }
    Vektor3d SaberiSa(const Vektor3d &v) {
        x += v.x; y += v.y; z += v.z;
        return *this;
    }
};
```

Primijetimo kako je ovom prilikom pokazivač "`this`" iskorišten za vraćanje izmijenjenog objekta iz metode (ne zaboravimo da dereferencirani "`this`" pokazivač predstavlja upravo objekat nad kojim je metoda pozvana). Sljedeća sekvenca naredbi dovodi do očekivanog ispisa (tj. ispisa "{15,20,25}" i "{22,23,27}"), tako da *izgleda* da prethodno rješenje radi korektno:

```
Vektor3d v1, v2;
v1.Postavi(3, 4, 5);
v2.Postavi(7, 3, 2);
v1.PomnoziSaSkalarom(5).Ispisi();           // {15,20,25}
std::cout << std::endl;
v1.SaberiSa(v2).Ispisi();                   // {22,23,27}
```

Međutim, sljedeći primjer će nas uvjeriti da nije baš sve u potpunosti u skladu s očekivanjima. Naime, mada bi se moglo očekivati da će sljedeća sekvenca naredbi dva puta ispisati "{22,23,27}", biće jedanput ispisano "{22,23,27}", a drugi put "{15,20,25}". Drugim riječima, što se tiče drugog ispisa, program se ponaša kao da se nad vektorom "`v1`" izvršila samo metoda "`PomnoziSaSkalarom`" a ne i metoda "`SaberiSa`":

```
Vektor3d v1, v2;
v1.Postavi(3, 4, 5);
v2.Postavi(7, 3, 2);
v1.PomnoziSaSkalarom(5).SaberiSa(v2).Ispisi(); // {22,23,27}
std::cout << std::endl;
v1.Ispisi();                                 // {15,20,25} ???
```

Objašnjenje ove prividne zagonetke leži u činjenici da se nad vektorom "`v1`" zaista izvršila samo metoda "`PomnoziSaSkalarom`" a ne i metoda "`SaberiSa`". Međutim, to odmah nameće nova pitanja *zašto je*

tako i kako je onda prvi ispis u skladu s očekivanjima. Odgovor na ova pitanja je u načinu *kako se vrši vraćanje rezultata iz funkcija*, odnosno *prenos rezultata funkcija nazad na mjesto poziva*. Naime, u normalnim okolnostima, kada god s **"return"** naredbom vratimo neku *vrijednost* iz funkcije, time se zapravo stvara novi (bezimeni) objekat čiji tip odgovara tipu povratne vrijednosti, u koji se smješta vrijednost izraza koji je naveden iza naredbe **"return"** i koji predstavlja rezultat funkcije. Ovdje je važno da shvatimo da kada se iza naredbe **"return"** nađe *ime nekog objekta*, ono što će biti vraćeno iz funkcije *nije stvarno taj objekat*, nego *novi objekat* koji je *njegova identična kopija*. Ovo je neophodno zbog toga što stvarni objekat može i da prestane postojati nakon završetka funkcije, npr. ukoliko on predstavlja lokalnu promjenljivu deklariranu unutar funkcije. Ovakvo ponašanje nije svojstveno samo klasama i funkcijama članicama klasa, nego predstavlja *opće pravilo kako se ma kakve vrijednosti vraćaju iz ma kakve funkcije*. Mada o ovakvom ponašanju nismo do sada posebno razmišljali, ono nam nikada do sada nije smetalo. Međutim, razmotrimo u ovom kontekstu značenje sljedeće konstrukcije:

```
v1.PomnoziSaSkalarom(5).Saberisa(v2).Ispisi();
```

U ovom slučaju, konstrukcija **"v1.PomnoziSaSkalarom(5)"** množi vektor **"v1"** s brojem **"5"** (što se odražava na sadržaj vektora **"v1"** koji poprima vrijednost **"{15,20,25}"**), i vraća kao rezultat *vrijednost* izmijenjenog vektora. Međutim, vraćeni vektor nije sam objekat **"v1"**, nego neki privremeni bezimeni objekat tipa **"Vektor3d"**, koji je njegova potpuno identična kopija! Na tu kopiju (koja također ima vrijednost **"{15,20,25}"**) primijenjuje se metoda **"Saberisa(v2)"** koja mijenja vrijednost tog bezimenog vektora (koja sad postaje **"{22,23,27}"**) i vraća kao rezultat *novi bezimeni vektor* koji ima vrijednost **"{22,23,27}"**. Nad tim bezimenim vektorom primijenjuje se metoda **"Ispisi()"**, koja naravno ispisuje **"{22,23,27}"**. Međutim, poenta je da se samo prva metoda **"PomnoziSaSkalarom(5)"** zaista izvršila nad vektorom **"v1"**, dok su se sve ostale metode izvršile nad nekim bezimenim vektorima koje predstavljaju kopije objekata koje su vraćene iz funkcije! Stoga vektor **"v1"** zadržava vrijednost kakvu je imao nakon primjene metode **"PomnoziSaSkalarom(5)"**, odnosno **"{15,20,25}"**. Zbog toga će sljedeći poziv **"v1.Ispisi()"** ispisati upravo **"{15,20,25}"**.

Činjenica da se iz funkcije uvijek vraća *kopija* objekta koji je naveden iza **"return"** naredbe a ne sam objekat, nije nam do sada smetala (naprotiv, bila je veoma korisna), s obzirom da nikada do sada nismo imali potrebu da nad objektom vraćenim iz funkcije primijenimo neku akciju koja bi trebala da promijeni sam vraćeni objekat. Međutim, kao što vidimo, uvođenjem *klasa* omogućeno je definiranje takvih objekata nad kojima se mogu primjenjivati akcije koje mogu *mijenjati* sadržaj objekta (tj. metode mutatori). Stoga, ovakav način vraćanja vrijednosti iz funkcija može napraviti probleme ukoliko se metode mutatori pozivaju *ulančano* (tj. kad se na rezultat jedne metode ponovo primjenjuje druga metoda). Šta da se radi? Izlaz iz ove situacije je veoma jednostavan: metode **"PomnoziSaSkalarom"** i **"Saberisa"** treba da umjesto samog objekta tipa **"Vektor3d"**, koji je *kopija* objekta koji želimo da vratimo, vrate *referencu* na objekat tipa **"Vektor3d"** koja predstavlja *alternativno ime* za objekat koji vraćamo, odnosno koja se *u potpunosti poistovjećuje* s objektom koji vraćamo. Stoga je u klasi **"Vektor3d"** dovoljno izvršiti samo neznatnu izmjenu, kao u sljedećoj deklaraciji:

```
class Vektor3d {  
    ... // Ovdje treba umetnuti definicije atributa i preostalih metoda...  
    Vektor3d &PomnoziSaSkalarom(double s) {  
        x *= s; y *= s; z *= s;  
        return *this; // Vraća se referenca!  
    }  
    Vektor3d &Saberisa(const Vektor3d &v) {  
        x += v.x; y += v.y; z += v.z;  
        return *this; // Vraća se referenca!  
    }  
};
```

Nakon ove izmjene, ulančani poziv poput **"v1.PomnoziSaSkalarom(5).Saberisa(v2)"** će zaista raditi u skladu s očekivanjima. Prvi poziv metode **"PomnoziSaSkalarom(5)"** izmijenjuje vektor **"v1"** i vratiti kao rezultat referencu na izmijenjeni vektor **"v1"** koja se u potpunosti poistovjećuje s njim, tako da će sljedeći poziv metode **"Saberisa(v2)"** zaista djelovati na vektor **"v1"**, što smo i željeli da postignemo.

Vraćanje reference na izmijenjeni objekat *veoma je uobičajeno ponašanje mutatorskih funkcija*, i neki čak savjetuju *da bi se tako trebale ponašati praktično sve mutatorske funkcije*, osim ukoliko nije jasno da bi one trebale da vrte nešto drugo kao rezultat (uglavnom, one nikada ne bi trebale da budu **"void"** funkcije). Da bismo uvidjeli razlog za to, pretpostavimo da u klasi **"Vektor3d"** postoje i tri mutatorske funkcije **"PostaviX"**, **"PostaviY"** i **"PostaviZ"**. Neka je sad potrebno promijeniti vrijednosti *x* i

y koordinate vektora "v" recimo na vrijednosti 5 i 3, bez mijenjanja koordinate z. Ukoliko bi ove tri funkcije bile funkcije koje ne vraćaju ništa (tj. "void" funkcije), to bismo morali uraditi ovako:

```
v.PostaviX(5); v.PostaviY(3);
```

Dakle, bile bi nam potrebne dvije naredbe. Međutim, kada bi ove tri funkcije vraćale referencu na izmijenjeni vektor, i dalje bi radio prethodni primjer (pri čemu bi povratna vrijednost funkcija bila prosto ignorirana), ali bi sada bilo moguće koristiti i *kaskadno pozivanje*, odnosno konstrukcija poput

```
v.PostaviX(5).PostaviY(3);
```

Zaista, poziv funkcije "PostaviX" nad objektom "v" vratiće referencu na izmijenjeni objekat "v" (koja se ponaša identično kao da je ona baš objekat "v") na koju se dalje primjenjuje funkcija "PostaviY" (odnosno ona se izvodi nad objektom "v"). Upravo je mogućnost kaskadnog pozivanja mutatorskih funkcija glavni razlog zašto bi se one trebale izvoditi tako da vraćaju referencu na izmijenjeni objekat.

U prethodnim primjerima smo koristili sintaksu poput "v1.SaberiSa(v2)" da saberemo vektore "v1" i "v2". Pri tome, funkcija "Saberi" je funkcija *mutator*, odnosno ona *mijenja* sam vektor "v1", tako da ona ima dejstvo koje bismo logički mogli opisati izrazom "v1 += v2" (zanemarimo to što zasad operator "+=" ne možemo primjenjivati na objekte tipa "Vektor3d"). Mada je ova sintaksa u potpunosti u duhu objektno orijentirane filozofije, sabiranje vektora bi bilo prirodnije izraziti čisto proceduralnom sintaksom poput "ZbirVektora(v1, v2)", pri čemu funkcija "ZbirVektora" ne bi modificirala ni vektor "v1" ni "v2", nego bi prosto vratila kao rezultat *novi vektor* koji je njihov zbir (tj. logički bi djelovala poput izraza "v1 + v2" a ne "v1 += v2". Ovo je pogotovo značajno u kontekstu dodjele rezultata nekom trećem vektoru, recimo u izrazu poput "v3 = ZbirVektora(v1, v2)" (alternativni izraz poput "v3 = v1.SaberiSa(v2)" trenutno ima nezgodnu propratnu posljedicu *izmjenu* vektora "v1"). Međutim, treba imati na umu da time što smo definirali tip "Vektor3d" kao klasu nismo izgubili pravo da pišemo *obične funkcije* (koje nisu funkcije članice) koje kao parametre primaju objekte tipa "Vektor3d" i koje vraćaju objekte tipa "Vektor3d" kao rezultat. Stoga nam niko ne brani da napišemo običnu funkciju "ZbirVektora" koju ćemo upravo pozivati na gore navedeni način (tj. kao standardnu funkciju, a ne primijenjenu nad nekim objektom). Mada izvjesni radikalisti smatraju da pisanje ovakvih funkcija nije u duhu objektno orijentiranog programiranja, većina ipak ima mišljenje da pisanje ovakvih funkcija u *umjerenoj količini* ne šteti objektno orijentiranoj filozofiji, pogotovo ukoliko vodi ka intuitivno prirodnijoj sintaksi. Stoga ćemo napisati funkciju "ZbirVektora" koja će koristiti gore opisanu sintaksu. Pri tome, moramo voditi računa da tako napisana funkcija neće imati privilegiju pristupa privatnim atributima klase "Vektor3d" (upravo zbog činjenice da ona nije funkcija članica nego obična funkcija). Zbog toga ćemo, dok ne upoznamo bolje rješenje, ovu funkciju napisati na indirektan način, koristeći funkcije članice "Postavi", "DajX", "DajY" i "DajZ" pomoću kojih ipak možemo posredno pristupiti atributima klase "Vektor3d":

```
Vektor3d ZbirVektora(const Vektor3d &v1, const Vektor3d &v2) {  
    Vektor3d v3;  
    v3.Postavi(v1.DajX() + v2.DajX(), v1.DajY() + v2.DajY(), v1.DajZ() + v2.DajZ());  
    return v3;  
}
```

Neko bi mogao prigovoriti da i sintaksa poput "v.DajDuzinu()" za nalaženje dužine vektora "v" nije posve prirodna, već da bi bilo prirodnije koristiti sintaksu poput "Duzina(v)". Međutim, ova primjedba je manje osnovana nego u slučaju sabiranja i već donekle kviri duh objektno orijentirane filozofije. Naime, i dužina vektora i njegove koordinate su izvjesne informacije *o vektoru* kojima je dobro pristupati na jednoobrazan način, bez obzira što su u konkretnoj implementaciji koordinate vektora izvedene kao njegovi atributi, dok se dužina posredno *računa* na osnovu koordinata. Već smo rekli da to korisnik klase *ne treba da zna*, tim prije što su sasvim moguće i drugačije implementacije za koje ovo ne vrijedi. S druge strane, sintaksa poput "Duzina(v)" središte pažnje prenosi na funkciju "Duzina", pri čemu se jasno naglašava da dužina vektora nije njegovo *svojstvo* nego se ona *računa* posredstvom funkcije "Duzina" koja kao *argument* uzima vektor a daje kao *rezultat* njegovu *dužinu*. Vidimo da se radi o klasičnom proceduralnom razmišljanju. Ipak, radi demonstracije nekih činjenica napisaćemo i *običnu funkciju* "Duzina" koja prima kao parametar vektor, a vraća njegovu dužinu, tako da se može koristiti sintaksa i "Duzina(v)" (ovako nešto imali smo kod tipa "complex", gdje smo ravnopravno mogli koristiti izraze poput "z.real()" i "real(z)"):

```
double Duzina(const Vektor3d &v) {  
    return v.DajDuzinu();  
}
```

Vidimo da je definicija funkcije "Duzina" trivijalna, jer smo za njenu izvedbu prosto iskoristili metodu "DajDuzinu" nad objektom koji joj je proslijeđen kao parametar, koja upravo radi ono što nam i treba. Interesantno je da mogu postojati i funkcija članica i obična funkcija s istim imenom, bez opasnosti da ih kompajler pomiješa, s obzirom da se one *razlikuju po načinu poziva* (funkcije članice se uvijek pozivaju nad nekim objektom, osim eventualno iz druge funkcije članice iste klase). Ipak, ukoliko bismo iz neke funkcije članice željeli pozvati *istoimenu* običnu funkciju, morali bismo ispred njenog imena pri pozivu dodati prefiks "::" (bez navođenja imena ikakve klase ispred). Na primjer, funkciju "Duzina" bismo iz istoimene funkcije članice morali pozivati s "::Duzina(v)", jer bi u suprotnom bilo shvaćeno da ta funkcija članica poziva samu sebe (tj. kao rekurzija)!

Vratimo se na implementaciju funkcije "ZbirVektora". Možemo reći da smo imali sreće što klasa "Vektor3d" ima metode "DajX", "DajY" i "DajZ", koje ipak omogućavaju da jednostavno pristupimo vrijednostima koordinata vektora. Da nismo imali ove metode nego da smo morali koristiti metodu "Ocitaj", implementacija funkcije "ZbirVektora" bila bi mnogo komplikovanija:

```
Vektor3d ZbirVektora(const Vektor3d &v1, const Vektor3d &v2) {  
    double v1_x, v1_y, v1_z, v2_x, v2_y, v2_z;  
    v1.Ocitaj(v1_x, v1_y, v1_z); v2.Ocitaj(v2_x, v2_y, v2_z);  
    Vektor3d v3;  
    v3.Postavi(v1_x + v2_x, v1_y + v2_y, v1_z + v2_z);  
    return v3;  
}
```

Da nije postojala ni metoda "Ocitaj", ruke bi nam bile potpuno vezane i uopće ne bismo bili u stanju napisati ovu funkciju. S druge strane, najprirodnija verzija funkcije "ZbirVektora" mogla bi se napisati kada bi ova funkcija *imala pravo pristupa privatnim atributima* klase "Vektor3d". Ona bi se tada mogla napisati prosto ovako (slično istoimenoj funkciji s ranijih predavanja koju smo pisali dok je tip "Vektor3d" bio samo *struktura*, a ne *klasa s privatnim atributima*):

```
Vektor3d ZbirVektora(const Vektor3d &v1, const Vektor3d &v2) {  
    Vektor3d v3;  
    v3.x = v1.x + v2.x; v3.y = v1.y + v2.y; v3.z = v1.z + v2.z;  
    return v3;  
}
```

Na svu sreću, tako nešto je *moгуće*. Naime, unutar deklaracije klase moguće je izvjesne obične funkcije (koje nisu funkcije članice klase) proglasiti *prijateljima klase*, čime one dobijaju pravo pristupa privatnim atributima i metodama te klase. Neku funkciju je moguće proglasiti prijateljem klase tako što se unutar deklaracije klase navede ključna riječ "**friend**", a zatim prototip funkcije koju proglašavamo prijateljem. Ovim funkcija ne postaje funkcija članica (tj. i dalje se poziva kao obična funkcija), samo dobija pravo pristupa privatnim dijelovima klase. Slijedi deklaracija klase "Vektor3d" koja proglašava funkciju "ZbirVektora" prijateljem klase, što omogućava njenu izvedbu na gore prikazani način:

```
class Vektor3d {  
    double x, y, z;  
public:  
    void Postavi(double x, double y, double z) {  
        Vektor3d::x = x; Vektor3d::y = y; Vektor3d::z = z;  
    }  
    void Ocitaj(double &x, double &y, double &z) const {  
        x = Vektor3d::x; y = Vektor3d::y; z = Vektor3d::z;  
    }  
    void Ispisi() const {  
        std::cout << "{" << x << ", " << y << ", " << z << " }";  
    }  
    double DajX() const { return x; }  
    double DajY() const { return y; }  
    double DajZ() const { return z; }  
    double DajDuzinu() const { return std::sqrt(x * x + y * y + z * z); }  
    Vektor3d &PomnoziSaSkalarom(double s) {  
        x *= s; y *= s; z *= s;  
        return *this;  
    }  
    Vektor3d &SaberiSa(const Vektor3d &v) {  
        x += v.x; y += v.y; z += v.z;  
        return *this;  
    }  
    friend Vektor3d ZbirVektora(const Vektor3d &v1, const Vektor3d &v2);  
};
```


Bitno je uočiti smisao ključne riječi **"friend"**. Bez navođenja ove ključne riječi, prototip funkcije **"ZbirVektora"** bio bi shvaćen kao prototip *funkcije članice* koja se primjenjuje nad nekim objektom tipa **"Vektor3d"**, a kojoj se pored toga još prenose dva vektora kao parametri (tj. funkcije članice koja bi se koristila u formi poput **"v3.ZbirVektora(v1, v2)"** gdje su **"v1"**, **"v2"** i **"v3"** neki objekti tipa **"Vektor3d"**. Tako, ključna riječ **"friend"** zapravo govori da se ne radi o funkciji članici nego o običnoj funkciji, ali kojoj se daju *sva prava po pitanju pristupa privatnoj sekciji klase* kakvu imaju i funkcije članice klase. Takve funkcije nazivamo *prijateljske funkcije* (engl. *friend functions*) ili *funkcije prijatelji klase*.

Treba naglasiti da se, bez obzira na prijateljski status funkcije **"ZbirVektora"**, ona (barem zasad) *nije mogla napisati ovako*, što je moglo kada je **"Vektor3d"** bila struktura a ne klasa:

```
Vektor3d ZbirVektora(const Vektor3d &v1, const Vektor3d &v2) {  
    Vektor3d v3{v1.x + v2.x, v1.y + v2.y, v1.z + v2.z};  
    return v3;  
} // OVO NE RADI!
```

Ne bi radilo ni ovo, što je također ranije radilo sa strukturama:

```
Vektor3d ZbirVektora(const Vektor3d &v1, const Vektor3d &v2) {  
    return {v1.x + v2.x, v1.y + v2.y, v1.z + v2.z};  
} // NI OVO NE RADI!
```

Zašto ovo ne radi? Poenta je u tome da *ukoliko klasa ima makar jedan privatni atribut* (a svaka dobra klasa trebala bi imati ne jedan, nego bi svi atributi trebali biti privatni), *inicijalizacija njenih primjeraka pomoću liste inicijalizatora postaje zabranjena* (mada ćemo uskoro vidjeti da se ovaj problem može prevazići definiranjem odgovarajućih tzv. *konstruktor*a, pomoću kojih se može, između ostalog, definirati kako će se tačno izvoditi inicijalizacija primjeraka klase putem listi inicijalizatora). Razlog za tu zabranu vrlo je jasan i logičan. Naime, kada bi takva inicijalizacija bila moguća, putem nje bismo prosto mogli zaobići sve zaštitne mehanizme koje smo uveli da ostvarimo kontrolu nad dozvoljenim vrijednostima atributa i postaviti vrijednosti atributa na ma kakve vrijednosti!

Neka klasa može bilo koju funkciju proglasiti svojim prijateljem. Naravno, to "prijateljstvo" ima smisla jedino ukoliko ta funkcija ima neke veze s tom klasom, što se tipično dešava u tri slučaja: ako funkcija *prima parametre* koji su tipa te klase (ili pokazivača na tu klasu, ili reference na tu klasu), ako funkcija *vraća rezultat* koji je tipa te klase, ili ako funkcija *deklarira lokalnu varijablu* tipa te klase (naravno, moguće je da se dešava i sve ovo skupa, kao u primjeru navedene funkcije **"ZbirVektora"**).

Prijateljske funkcije se također mogu implementirati odmah unutar deklaracije klase, pri čemu se tada, slično kao u slučaju funkcija članice klase, one automatski proglašavaju za umetnute funkcije (tj. podrazumijeva se modifikator **"inline"**). Međutim, to treba *strogo izbjegavati*, s obzirom da postoje ozbiljni problemi s *vidljivošću* tako definiranih funkcija (tj. odakle se njima može pristupiti), što je nuspojava nekih vrlo kompleksnih pravila kako se u jeziku C++ inače određuje vidljivost pojedinih identifikatora, naročito pravila poznatog kao *traženje ovisno o argumentu* ili skraćeno *ADL* (od engl. *argument dependent lookup*). Detaljno objašnjenje izlazi izvan opsega ovog kursa, ali recimo samo da su takve funkcije *praktično nedostupne za ostatak programa ukoliko im nijedan parametar nije tipa klase u kojoj se nalaze, osim ukoliko je negdje izvan klase naveden njihov prototip* (postoje doduše neke egzotične situacije u kojima programer baš to želi da postigne, tako da tada treba definirati prijateljske funkcije unutar klase, ali to spada u napredne trikove u koje nećemo ulaziti). Primjer problematične situacije imamo u sljedećem primjeru, gdje je prijateljska funkcija **"Problematicna"** implementirana unutar same definicije klase **"NekaKlasa"**, ali kako joj nijedan parametar nije tipa **"NekaKlasa"**, ona je praktično *nedostupna iz ostatka programa* (osim ukoliko negdje izvan deklaracije klase ne napišemo posebno i njen prototip):

```
class NekaKlasa {  
    int x;  
public:  
    friend NekaKlasa Problematicna(int x) {  
        NekaKlasa klasa; klasa.x = x; return klasa; // Nebitno šta funkcija radi...  
    }  
};  
...  
NekaKlasa k;  
k = Problematicna(10); // Ne radi, jer je funkcija "Problematicna" nedostupna!
```

Interesantno je napomenuti da postoji i alternativno rješenje kojim se postiže slična funkcionalnost kao pomoću prijateljskih funkcija, a u kojima se ipak ne koriste obične funkcije, nego samo funkcije članice. Ideja je da se umjesto običnih funkcija koriste *statičke funkcije članice*, koje samim tim što su funkcije članice svakako imaju pravo pristupa privatnim dijelovima klase. U konkretnom slučaju koji razmatramo, funkcija "ZbirVektora" bi, umjesto obične funkcije, postala statička funkcija članica:

```
class Vektor3d {  
    ...           // U ostatku klase ništa se ne mijenja  
    static Vektor3d ZbirVektora(const Vektor3d &v1, const Vektor3d &v2);  
};
```

Implementacija ove funkcije članice bila bi praktično identična kao i u slučaju kada smo koristili klasičnu funkciju (obratite pažnju samo na kvalifikator "Vektor3d::" u deklaraciji funkcije):

```
Vektor3d Vektor3d::ZbirVektora(const Vektor3d &v1, const Vektor3d &v2) {  
    Vektor3d v3;  
    v3.x = v1.x + v2.x; v3.y = v1.y + v2.y; v3.z = v1.z + v2.z;  
    return v3;  
}
```

Jedini nedostatak ovog rješenja je što sada umjesto konstrukcije " $v3 = \text{ZbirVektora}(v1, v2)$ " moramo koristiti rogobatniju konstrukciju " $v3 = \text{Vektor3d::ZbirVektora}(v1, v2)$ ". Neki smatraju da je ova sintaksa, bez obzira na rogobatnost, logičnija i više "u duhu" objektno zasnovanog programiranja, jer jasno ističe logičku pripadnost funkcije "ZbirVektora" klasi "Vektor3d". Zapravo, u ovom slučaju bila bi dobra ideja preimenovati naziv funkcije "ZbirVektora" prosto u "Zbir", jer već sam kvalifikator "Vektor3d::" jasno ističe da se radi o zbiru vektora a ne nečega drugog (odnosno konstrukcija poput "Vektor3d::ZbirVektora" sadrži stilsku grešku u lingvistici poznatu kao *pleonazam*), tako da bismo tada koristili prihvatljiviju konstrukciju poput " $v3 = \text{Vektor3d::Zbir}(v1, v2)$ ". Ipak, u jeziku C++ se češće koriste rješenja zasnovana na prijateljskim funkcijama. Ovdje prikazano rješenje koje koristi statičke funkcije članice uglavnom se koristi u čisto objektno orijentiranim jezicima kao što su Java, C# i srodni kod kojih klasične funkcije (ne-članice) *uopće ne postoje*.

Na prvi pogled, iz prethodnog primjera moglo bi se pogrešno zaključiti da su prijateljske funkcije zapravo *nepotrebne*, jer se gotovo ista funkcionalnost može ostvariti pomoću statičkih funkcija članica (samo uz neznatno drugačiju sintaksu). Ipak, taj zaključak je *pogrešan*. Statičke funkcije članice neke klase ipak *pripadaju toj klasi*, za razliku od prijateljskih funkcija koje *ne pripadaju nijednoj klasi*. Stoga bilo koja obična funkcija (nečlanica) može da u isto vrijeme bude *prijateljska funkcija za više od jedne klase*, dok statička funkcija članica mora pripadati isključivo jednoj klasi. Pretpostavimo, na primjer, da imamo klasu "Matrica" koja modelira neku matricu, i klasu "Vektor" koja modelira neki vektor (na ovom mjestu nije bitno kako bi se takve klase mogle implementirati, to ćemo naučiti kasnije). U linearnoj algebri je poznato da je moguće množiti matricu s vektorom (pod uvjetom da vektor ima onoliko elemenata koliko matrica ima redova), pri čemu se vektor tada tretira kao matrica sa samo jednom kolonom, dok se kao rezultat takvog množenja dobija ponovo vektor. Slijedi da bi bilo prirodno imati funkciju za množenje matrice s vektorom, čiji bi prototip izgledao poput sljedećeg:

```
Vektor ProizvodMatriceIVektora(const Matrica &m, const Vektor &v);
```

Jasno je da bi funkcija "ProizvodMatriceIVektora" trebala pristupati privatnim atributima i klase "Matrica" i klase "Vektor", tako da bi bilo pametno deklarirati ovu funkciju kao prijateljsku funkciju u obje ove klase. Međutim, u jezicima koji ne podržavaju prijateljske funkcije, kao što su Java ili C#, morali bismo ovu funkciju staviti kao statičku ili u klasu "Matrica", ili u klasu "Vektor". Međutim, stavimo li je u klasu "Matrica", nećemo imati direktan pristup privatnim atributima klase "Vektor", a stavimo li je u klasu "Vektor", nećemo imati direktan pristup privatnim atributima klase "Matrica". Pored ovih očitih ograničenja, javlja se i logički problem pripadnosti, jer teško je smatrati da ova funkcija pripada samo jednoj od ove dvije klase — ona na izvjestan način "pripada" i jednoj i drugoj klasi. U tome je osnovna prednost koju nude prijateljske funkcije u odnosu na statičke funkcije članice.

Dugo vremena su se vodile intenzivne filozofske rasprave o tome da li prijateljske funkcije štete objektno orijentiranoj filozofiji. Jedan argument protiv njihove upotrebe je bio da one *narušavaju skrivanje podataka*. To mišljenje je odbačeno zbog činjenice da nijedna funkcija sama sebe ne može proglasiti prijateljem neke klase (i na taj način neovlašteno pristupiti njenim atributima), već upravo sama klasa definira kojim funkcijama "može vjerovati". Ako klasa "ima povjerenje" u svoje funkcije

članice, zašto ne bi mogla imati povjerenje i u obične funkcije koje sama specificira? Drugi argument protiv njih je da prijateljske funkcije *podstiču korištenje sintakse koja nije u duhu objektno orijentiranog programiranja*. Ovo je doduše tačno, ali je činjenica da objektno orijentiranu filozofiju ne čini *sintaksa*, već četiri osnovna postulata: *skrivanje podataka, enkapsulacija, nasljeđivanje i polimorfizam*. Objektno orijentirana sintaksa može nam pomoći da razmišljamo na objektno orijentiran način, ali sama za sebe ne čini objektno orijentiranu filozofiju. Program koji poštuje postulate objektno orijentirane filozofije neće postati neobjektno orijentiran samo usljed toga što se s vremena na vrijeme koristi sintaksa koja nije strogo objektno orijentirana, pogotovo u slučajevima kad je ta sintaksa prirodnija. Što se tiče postulata objektno orijentirane filozofije, prijateljske funkcije ne narušavaju ni enkapsulaciju, jer se proglašenje funkcije prijateljskom također može smatrati *dijelom interfejsa klase* (tj. opisom šta se s klasom može raditi) a ne nečim posve odvojenim od klase, dok ćemo kasnije vidjeti da prijateljske funkcije ne narušavaju ni preostala dva koncepta: nasljeđivanje i polimorfizam. Dakle, nema mjesta tvrdnjama da su prijateljske funkcije ikakva smetnja objektno orijentiranom pristupu programiranju.

Postoje čak i mnogi slučajevi u kojima insistiranje na čisto objektno orijentiranoj sintaksi može dovesti do zabune, tako da je u takvim slučajevima bolje koristiti klasične funkcije i proceduralnu sintaksu (ovim se ne mora nužno pokvariti filozofija objektno orijentiranog programiranja). Razmotrimo, na primjer, funkcije članice "PomnoziSaSkalarom" i "Saberisa" klase "Vektor3d". Ove funkcije su klasične metode mutatori koje mijenjaju objekat na koji djeluju, i vraćaju kao rezultat referencu na izmijenjeni objekat (uglavnom s ciljem da se podrže ulančani pozivi). Međutim, pretpostavimo da smo ove dvije metode izveli tako da *ne mijenjaju* vektor na koji djeluju, nego da *samo vraćaju kao rezultat* vektor koji je nastao množenjem vektora na koji djeluju sa skalarom, odnosno sabiranjem vektora na koji djeluju s drugim vektorom (drugim riječima, da smo ih izveli tako da se one logički ponašaju poput operatora "+" odnosno "*" a ne poput operatora "+=" odnosno "*="). Na taj način, ove metode bi promijenile svoj karakter, odnosno umjesto mutatora postale bi inspektori. Njihove implementacije uz takvu modifikaciju mogle bi izgledati recimo ovako (primijetimo da ovdje ne samo da ne vraćamo reference iz funkcija, nego reference ne bismo ni smjeli vratiti, s obzirom da je "v" lokalna promjenljiva, pa bismo u slučaju vraćanja reference dobili viseću referencu):

```
class Vektor3d {
...                               // Osim naredne dvije funkcije, sve ostaje isto...
Vektor3d PomnoziSaSkalarom(double s) const {
    Vektor3d v;                               // Novi vektor!
    v.x = x * s; v.y = y * s; v.z = z * s;
    return v;
}
Vektor3d Saberisa(const Vektor3d &v) const {
    Vektor3d v1;                               // Novi vektor!
    v1.x = x + v.x; v1.y = y + v.y; v1.z = z + v.z;
    return v1;
}
};
```

Ovakve metode se ne smatraju dobrim, zbog toga što mogu biti *zbunjujuće*. Na primjer, pogledajmo sljedeću naredbu, uz pretpostavku da imamo ovako definiranu klasu "Vektor3d" i dva primjerka "v1" i "v2" ove klase:

```
v1.Saberisa(v2);                               // Ovo ne proizvodi nikakav krajnji efekat!
```

Šta radi ova naredba? Efektivno gledano, *ama baš ništa*. Metoda "Saberisa" ne utiče na vektor "v1" na koji je primijenjena, već *samo vraća kao rezultat* vektor koji je nastao sabiranjem vektora "v1" s vektorom "v2". Međutim, vraćena vrijednost iz ove metode se ne koristi nizašta (tj. ignorira se), tako da, na kraju, ova naredba ne ostvaruje *nikakav efekat* (isto kao kad bismo imali dvije cjelobrojne promjenljive "x" i "y" i zadali izraz "x + y" kao *samostalnu naredbu*, što je posve *legalno* ali *besmisleno*). Ovakvo definirana metoda bi imala smisla jedino ukoliko se njen rezultat na neki način *iskoristi*, na primjer u nekoj dodjeli poput "v1 = v1.Saberisa(v2)". Naravno, vraćeni rezultat smo mogli dodijeliti i nekom drugom vektoru, a ne ponovo vektoru "v1". Ipak, ovakva metoda je više *zbunjujuća* nego korisna. Njeno ime "Saberisa" i sintaksa upućuju na to da će sadržaj vektora "v1" biti izmijenjen, a zapravo neće biti. Ranije napisana funkcija "ZbirVektora" je *mnogo jasnija*, tim prije što ona oba svoja parametra tretira *ravnopravno*. Ista primjedba bi vrijedila za izmijenjenu verziju metode "PomnoziSaSkalarom".

Razmotrimo još jedan primjer. Klasa "Datum" ima metodu "PredjiNaSljedećiDan" koja je *mutator*, jer *modificira* objekat na koji je primijenjena, tako da on nakon modifikacije sadrži sljedeći datum po

kalendaru. Ukoliko bismo ovu metodu izmijenili tako da ne utiče na datum na koji je primjenjena nego da umjesto toga *vraća kao rezultat* sljedeći datum po kalendaru, ona bi postala *vrlo zbunjujuća*, zbog istog razloga kao u prethodnom primjeru. Ako baš želimo to uraditi, onda bi joj bolje bilo promijeniti ime u "*DajSljedećiDan*". Međutim, još bolje je za tu svrhu upotrijebiti *običnu funkciju*, koju možemo deklarirati kao prijateljsku (s ciljem dobijanja prava pristupa privatnim atributima klase). Takva funkcija mogla bi se implementirati recimo ovako:

```
Datum SljedećiDan(const Datum &dat) {
    Datum dat1 = dat; // Novi datum!
    dat1.dan++;
    if(dat1.dan > Datum::BrojDanaUMjesecu(mjesec, godina)) {
        dat1.dan = 1; dat1.mjesec++;
    }
    if(dat1.mjesec > 12) {
        dat1.mjesec = 1; dat1.godina++;
    }
    return dat1;
}
```

Jasno je šta se ovdje dešava: ova funkcija prvo kreira *novi objekat* "*dat1*" tipa "*Datum*" koji je *kopija* parametra "*dat*", nakon čega se ta kopija modifikira i vraća kao rezultat.

Iz izloženih primjera možemo zaključiti sljedeće: *metode inspektori koje kao rezultat vraćaju objekat tipa klase u kojoj su definirane* (npr. metode inspektori u klasi "*Vektor3d*" koje kao rezultat vraćaju objekte tipa "*Vektor3d*") *mogu biti vrlo zbunjujuće*. Ukoliko ipak želimo definirati i takve metode, zbrku možemo smanjiti ukoliko im pažljivo izaberemo ime. Na primjer, u prethodnom primjeru, konfuziju bismo mogli osjetno smanjiti ukoliko bismo metodu koja vraća sljedeći datum umjesto "*PredjiNaSljedećiDan*" nazvali "*DajSljedećiDan*". Na taj način, samo ime metode sugerira da ona ne mijenja objekat na koji djeluje, već samo *vraća kao rezultat novi objekat* koji se dobija nakon obavljene transformacije. Slično, u slučaju modificirane klase "*Vektor3d*", zbrka bi mogla biti manja ukoliko bi se metode "*PomnoziSaSkalarom*" odnosno "*SaberiSa*" preimenovala u "*DajProduktSaSkalarom*" odnosno "*DajZbirSa*". Međutim, najbolje je takve metode uopće ne praviti, nego umjesto njih koristiti klasične funkcije, ukoliko nam je već potrebna takva funkcionalnost.

Ponekad je potrebno učiniti da se neka metoda neke druge klase učini prijateljem neke klase, tj. da joj se omogući pristup njenim privatnim atributima i metodama. To se može učiniti isto kao pri proglašenju obične funkcije prijateljskom, samo se ispred imena metode mora navesti i ime pripadne klase praćeno operatorom "::*". Ukoliko je potrebno da sve metode neke druge klase (npr. klase "B") imaju pristup privatnim atributima i metodama neke klase (npr. klase "A"), tada je najbolje u deklaraciju klase "A" staviti deklaraciju poput*

```
friend class B;
```

Na taj način se čitava klasa "*B*" proglašava prijateljem klase "*A*", te kažemo da klasa "*A*" ima klasu "*B*" za *prijateljsku klasu* (engl. *friend class*). To preciznije znači da sve metode klase "*B*" postaju prijatelji klase "*A*". Takvo "prijateljstvo" nije ni simetrično ni tranzitivno. Drugim riječima, prethodna deklaracija ne proglašava automatski klasu "*A*" prijateljem klase "*B*". Također, ako je klasa "*B*" prijatelj klase "*A*", a klasa "*C*" prijatelj klase "*B*", to ne čini automatski klasu "*C*" prijateljem klase "*A*".

Važno je istaknuti da nijedna funkcija ni klasa *ne može sama sebe proglasiti prijateljem neke klase*, nego *jedino klasa može nekoj funkciji ili klasi dati tu privilegiju* (da nije tako, mogli bismo lako zaobići sve zaštitne mehanizme). U suštini, prijateljske funkcije neke klase tipično piše sam autor klase, tj. ista osoba koja piše i njene funkcije članice, a ne korisnik same klase.

Svrha ovog predavanja je uvod u osnovne smjernice koje vode ka objektno zasnovanom odnosno objektno orijentiranom pristupu programiranju. Većina novih programera pri prvom susretu s ovim konceptima izražava sumnju u njihovu korist i postavlja pitanje da li je trud koji je potrebno uložiti u objektno orijentirano programiranje (s obzirom da ono prisiljava programera na veću disciplinu i razmišljanje unaprijed) isplativ. Da je odgovor na ovo pitanje itekako potvrđan, uvjerićemo se u narednim izlaganjima, u kojima će se jasno vidjeti sve prednosti tehnika objektno zasnovanog i objektno orijentiranog programiranja kroz razne postupke koji bi bili jako teško ostvarljivi držeći se samo filozofije proceduralnog programiranja.